

# Prompts

## When to use this

Use prompts when you want users to select from a list of discrete options, especially in command menus or configuration flows. Consoul prompts support default selections, per-option colours, and cancelation phrases such as “back” or “exit”.

## Key types

- `SelectionPrompt` – Base prompt for string-labelled options.
- `SelectionPrompt<T>` – Generic prompt that maps complex objects to display labels.
- `SelectOption` – Represents an individual option with colour and render style metadata.
- `PromptResult` – Returned by `SelectionPrompt.Render`, exposing `HasSelection`, `Index`, and `IsCanceled`.

## Minimal example

```
var prompt = new SelectionPrompt("Select a database migration to run");
prompt.Add("Initial schema", ConsoleColor.Green, isDefault: true);
prompt.Add("Add auditing", ConsoleColor.Yellow);
prompt.Add("Drop legacy tables", ConsoleColor.Red);

PromptResult result = prompt.Render();
if (result.IsCanceled)
{
    Consoul.Write("No migration executed", ConsoleColor.DarkYellow);
}
else if (result.HasSelection)
{
    switch (result.Index)
    {
        case 0:
            RunInitialMigration();
            break;
        case 1:
            RunAuditMigration();
            break;
        case 2:
            RunDropMigration();
            break;
    }
}
```

## Advanced tips

- **Default option** – Set `isDefault: true` on an option to accept it when the user presses enter without typing anything. Combine this with `PromptResult.HasSelection` to branch logic when users accept the default versus explicitly typing an index.
- **Cancellation phrases** – Users can type “back”, “exit”, or “go back” to cancel. Check `PromptResult.IsCanceled` instead of comparing against sentinel values, and honour cancellation inside your view or command loop. When wrapping prompts in `StaticView/DynamicView`, forward the cancel signal to `GoBack()` so navigation stays consistent.
- **Generic prompts** – Use `SelectionPrompt<T>` with a `labelSelector` to work directly with domain objects rather than indexes. After calling `Render()`, map `PromptResult.Index` back to the underlying collection to retrieve the chosen instance, or store the selection in a tuple (`PromptResult result, IReadOnlyList<T> options`) so you can match the index even if the source list mutates.
- **Custom render styles** – `OptionRenderStyle.Checkbox` is useful for multi-select style menus. Toggle `SelectOption.Selected` before rendering to show which entries are active even though the prompt captures a single choice, and update colours per option to convey validation status or dependencies.
- **Clearing vs. appending** – Toggle the `clear` constructor argument or `ClearConsole` property to control whether the prompt redraws the screen. This is helpful when prompts live inside views that manage their own buffers or when you want to preserve command history above the prompt.
- **Input preprocessing** – Derive from `SelectionPrompt` and override `Render` when you need to normalise user input—uppercase commands, trim whitespace, or support synonyms—before the base implementation attempts to parse selections.
- **Accessibility** – Provide numeric prefixes in labels (e.g., “1) Start”) so screen readers or copy/paste workflows can identify options quickly, and favour high-contrast colour schemes using `SelectOption.Color` for low-vision scenarios.